

Building Custom Hooks

Re-using Logic

components > Login > JS Login.js > [🔍] emailReducer

```
import React, { useState, useEffect, useReducer, useContext } from 'react';
```

```
import Card from '../UI/Card/Card';
```

```
import classes from './Login.module.css';
```

```
import Button from '../UI/Button/Button';
```

```
import AuthContext from '../../store/auth-context';
```

```
const emailReducer = (state, action) => {
```

```
  useState();
```

```
(alias) useState<undefined>(): [undefined, React.Dispatch<(prevState: undefined) => undefined>] (+1 overload)
```

```
import useState
```

Returns a stateful value, and a function to update it.

@version — 16.8.0

Rules of Hooks

Only call React Hooks in **React Functions**

React
Component
Functions

Custom Hooks
(covered later!)

Only call React Hooks at the **Top Level**

Don't call them
in nested
functions

Don't call them
in any block
statements

+ extra, unofficial Rule for **useEffect()**: ALWAYS add everything you refer to inside of `useEffect()` as a dependency!

Module Content

What & Why?

Building a Custom Hook

Custom Hook Rules & Practices

What are "Custom Hooks"?

Outsource **stateful** logic into **re-usable functions**



Unlike "regular functions", custom hooks can use other React hooks and React state

✓ REACT-COMPLETE-GUIDE

- > .vscode
- > node_modules
- > public
- ✓ src
 - > components
 - JS App.js
 - index.css
 - JS index.js
 - .eslintcache
 - .gitignore
 - package-lock.json
 - package.json
 - README.md

src > JS App.js > App > fetchTasks

```
6 function App() {
7   const [isLoading, setIsLoading] = useState(false);
8   const [error, setError] = useState(null);
9   const [tasks, setTasks] = useState([]);
10
11   const fetchTasks = async (taskText) => {
12     setIsLoading(true);
13     setError(null);
14     try {
15       const response = await fetch(
16         'https://react-http-6b4a6.firebaseio.com/tasks.json'
17       );
18
19       if (!response.ok) {
20         throw new Error('Request failed!');
21       }
22
23       const data = await response.json();
```

TERMINAL

PROBLEMS

OUTPUT

...

1: node

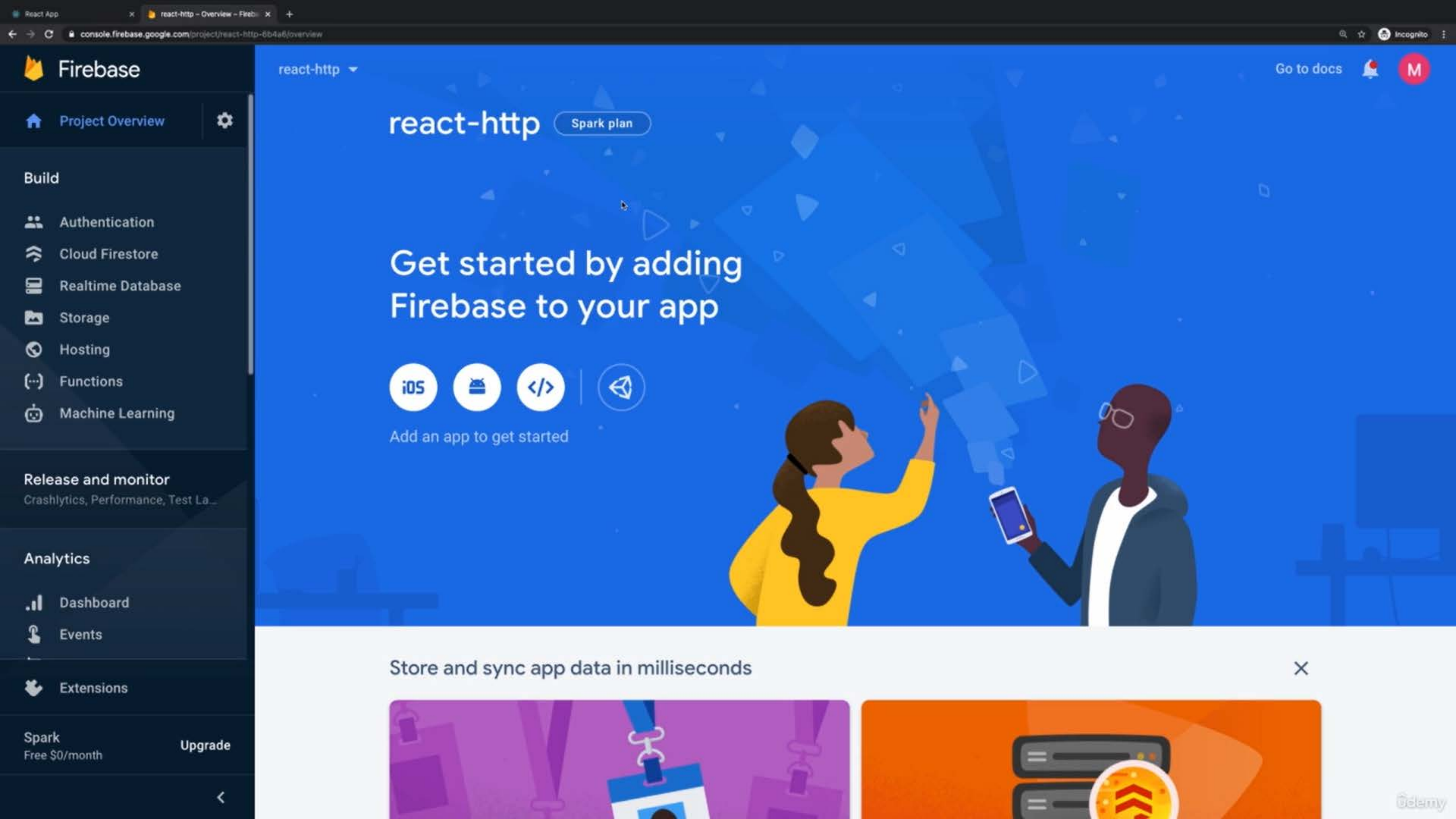
v

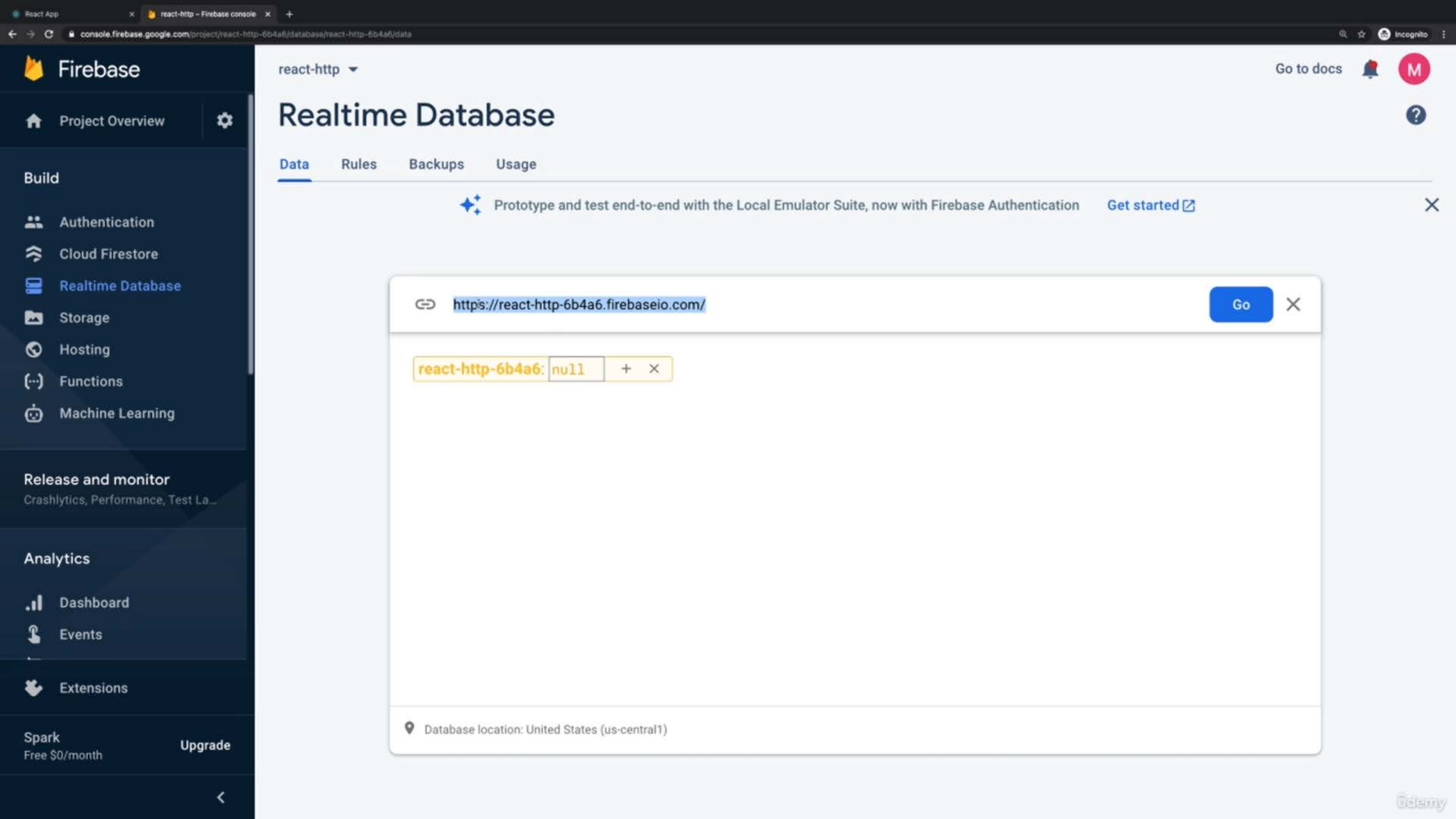
+

^

X

Note that the development build is not optimized.
To create a production build, use `npm run build`.





✓ REACT-COMPLETE-GUIDE

- > .vscode
- > node_modules
- > public
- ✓ src
 - > components
 - JS App.js
 - index.css
 - JS index.js
 - .eslintcache
 - .gitignore
 - package-lock.json
 - package.json
 - README.md

src > JS App.js > App > fetchTasks > response

```
function App() {  
  7   const [isLoading, setIsLoading] = useState(false);  
  8   const [error, setError] = useState(null);  
  9   const [tasks, setTasks] = useState([]);  
 10  
 11   const fetchTasks = async (taskText) => {  
 12     setIsLoading(true);  
 13     setError(null);  
 14     try {  
 15       const response = await fetch(  
 16         'https://react-http-6b4a6.firebaseio.com/tasks.json'  
 17       );  
 18  
 19       if (!response.ok) {  
 20         throw new Error('Request failed!');  
 21       }  
 22  
 23       const data = await response.json();  
 24     } catch (error) {  
 25       setError(error);  
 26     }  
 27   }  
 28 }  
 29 export default App;
```

TERMINAL

PROBLEMS

OUTPUT

...

1: node

v

+

^

X

Note that the development build is not optimized.
To create a production build, use `npm run build`.



Add Task

No tasks found. Start adding some!

Learn all about hooks!

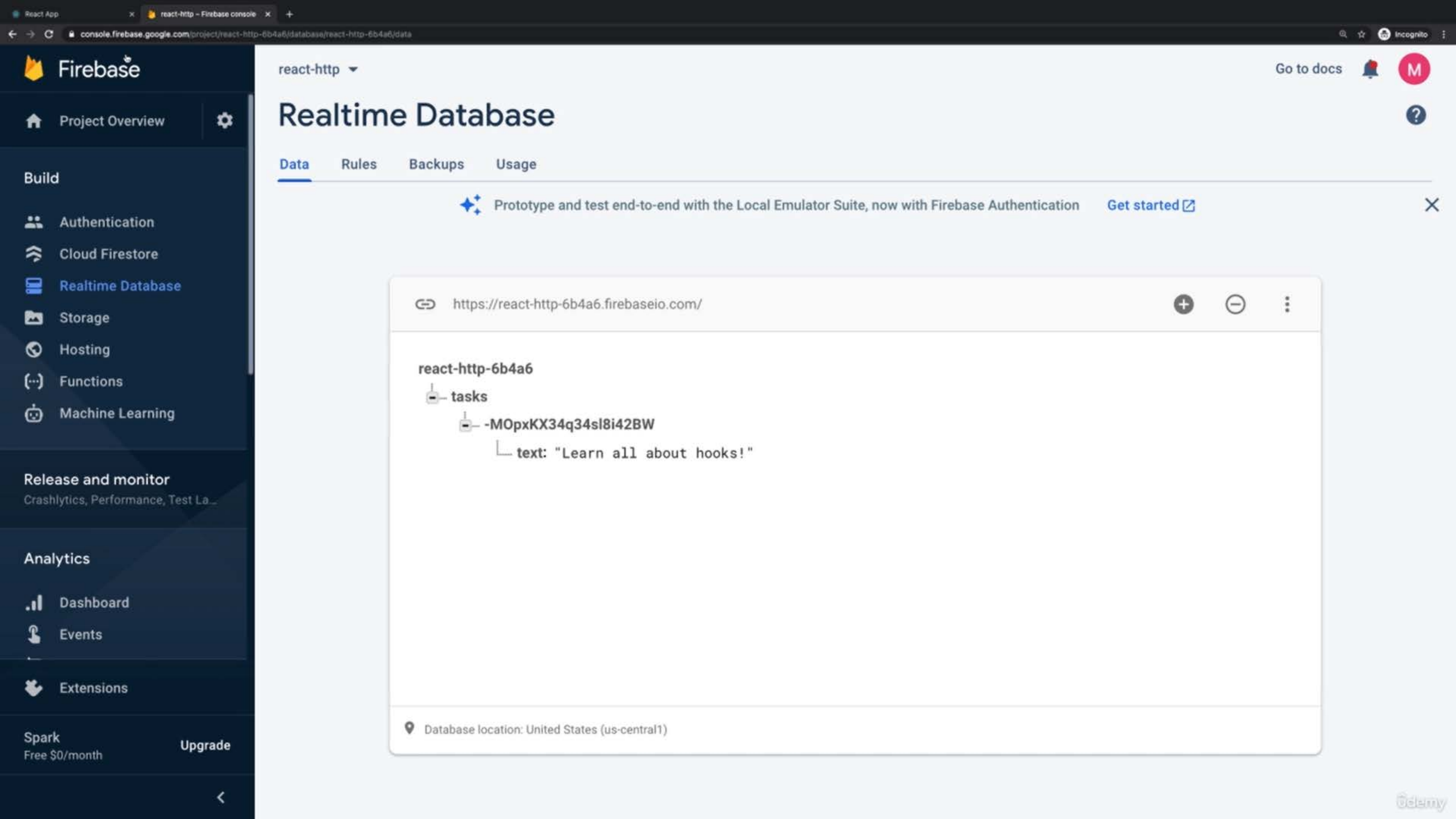
Add Task

No tasks found. Start adding some!

Learn all about hooks!

Add Task

Learn all about hooks!



Test

Add Task

Learn all about hooks!

Test